RNA-seq with Omics-Studio

and OMAKASE — from clicking apps to approving a recipe

From reads to changed genes · where an AI agent may help

Masaomi Hatakeyama · Functional Genomics Center Zurich · Bio610 · 28 October 2026

slides · ask about a slide

→ next · ↑ find a slide

Today: one RNA-seq analysis, from reads to genes

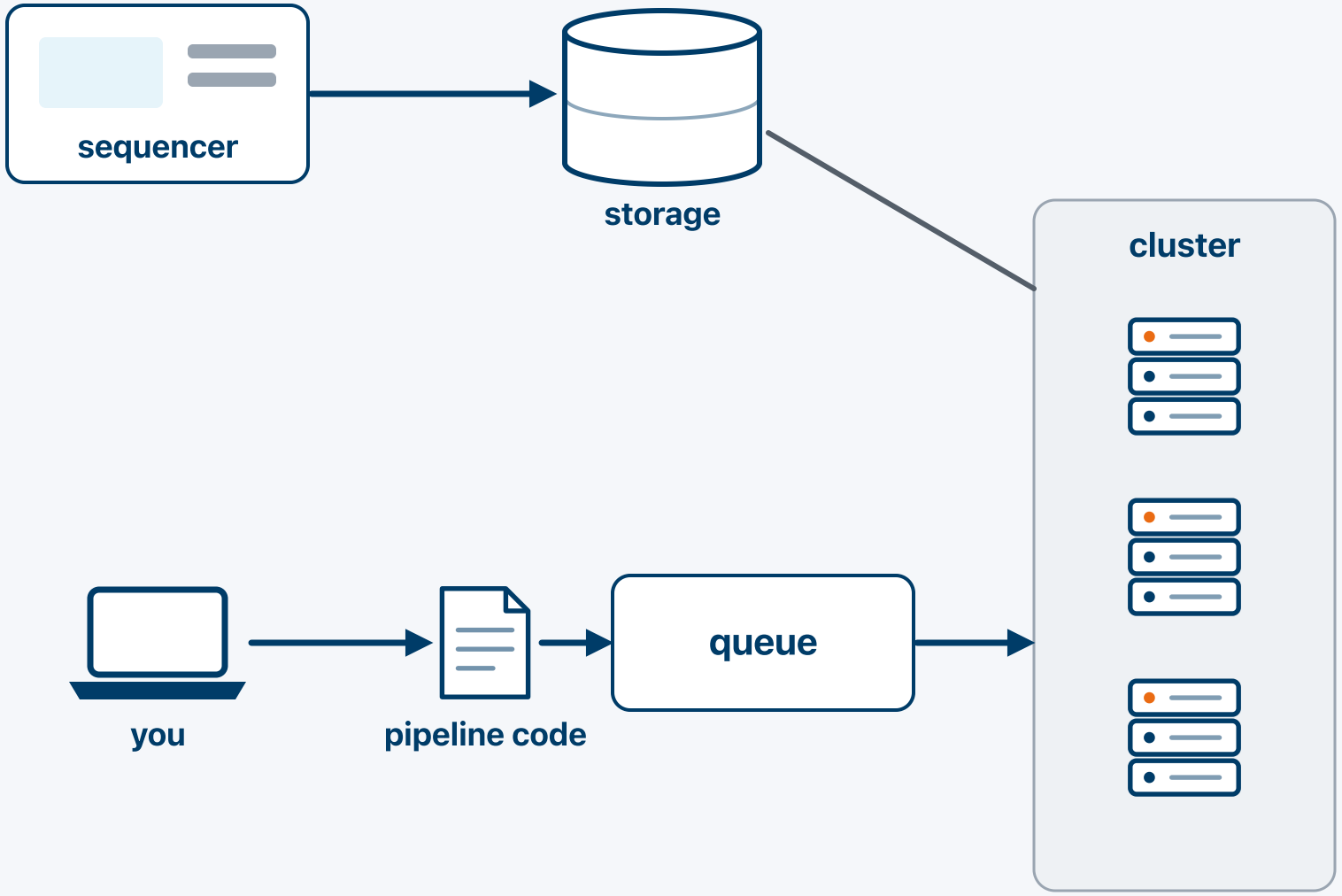
1. **RNA-seq: the steps**
   Part 1
2. **run it in Omics-Studio**
   Part 2 · hands-on
3. **OMAKASE: AI proposes, you approve**
   Part 3 · hands-on

Not today: sequencers, statistics

11.00–11.30	Lecture: Parts 1–2
11.30–11.45	Lecture: Part 3
11.45–12.00	Demo: Omics-Studio, OMAKASE
12.00–13.00	Lunch
13.00–13.35	Hands-on: RNA-seq
13.35–13.50	Hands-on: OMAKASE
13.50–14.00	Wrap-up

draft times

NGS data analysis needs a cluster and pipeline code



1 Too big

> 2 GB per sample
many CPU hours

2 Repeatable

same input,
same result

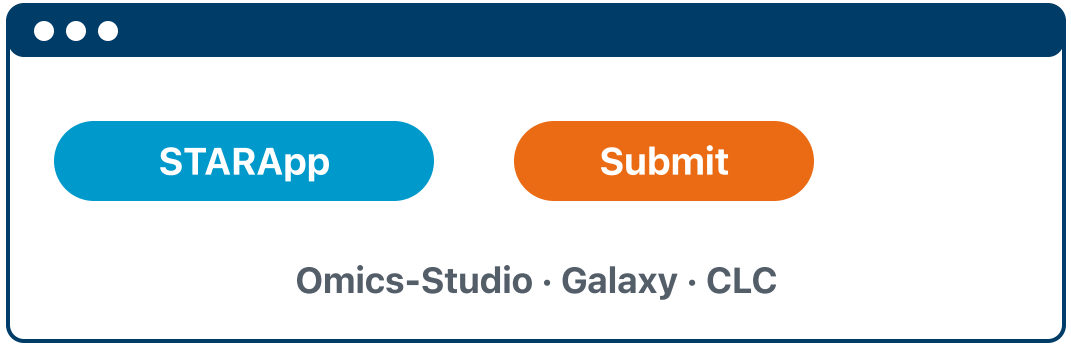
Scripts are exact; web tools are easier to start

Script

```
bwa index reference.fasta
bwa mem reference.fasta reads.fq > aln.sam
samtools sort -o aln.bam aln.sam
samtools index aln.bam
sbatch -n 8 --mem=100G map.sh
```

- exact, repeatable
- hard to learn

Web tool

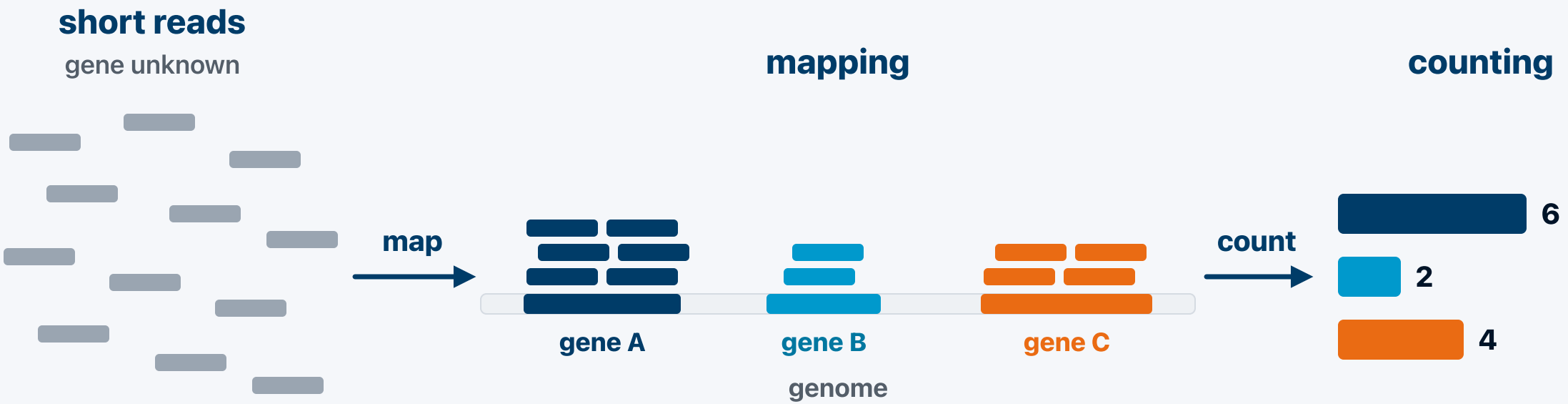


- easy to start
- only its tools

Omics-Studio: you click, it writes the script

deterministic: same choices, same script · no AI

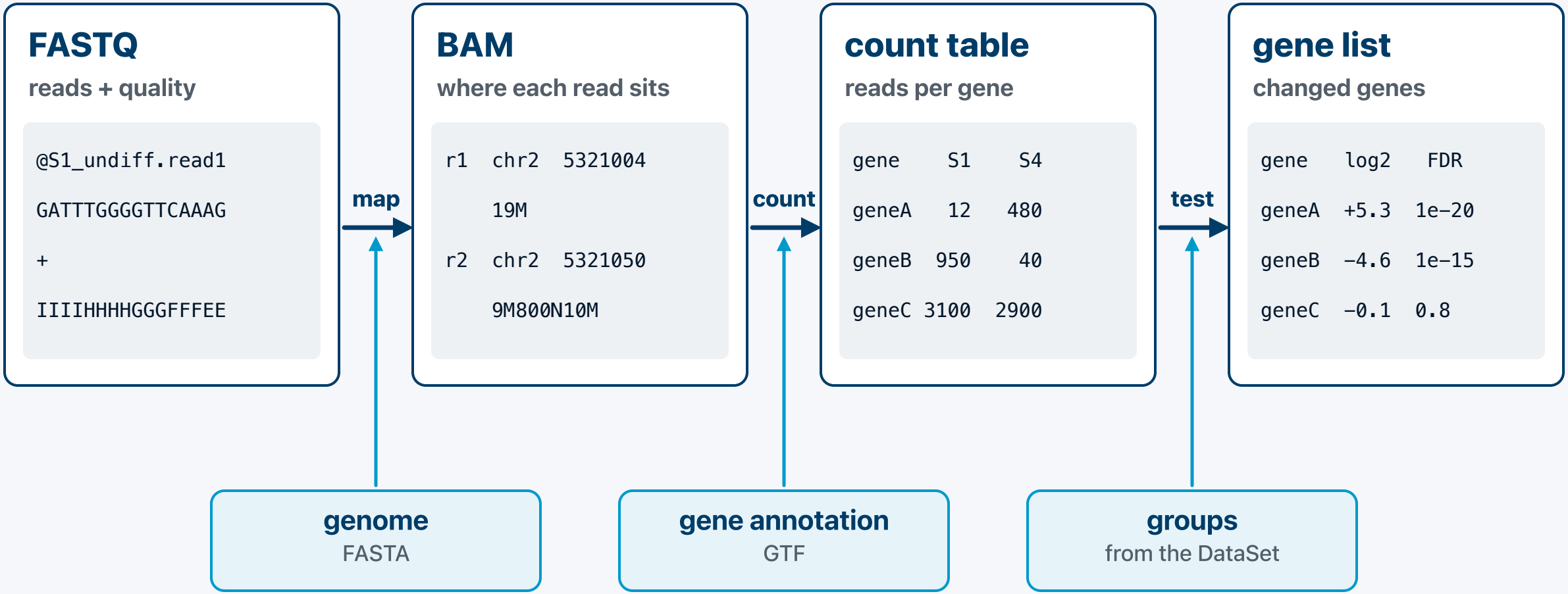
RNA-seq: place each read on the genome, then count



more reads = more RNA

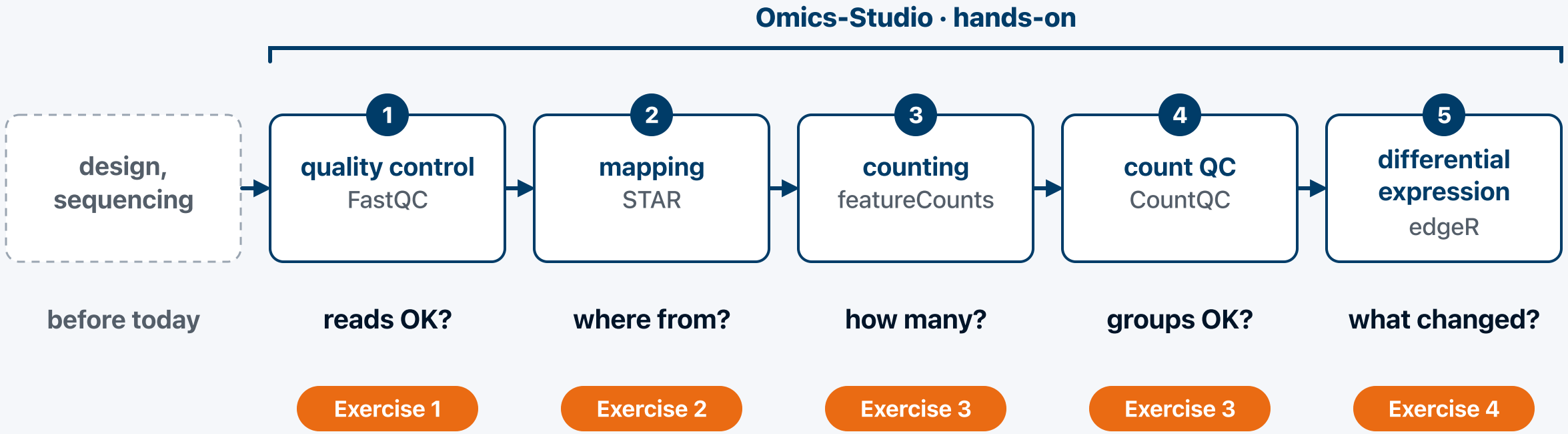
compare groups: which genes change

Each step writes a file that the next step reads

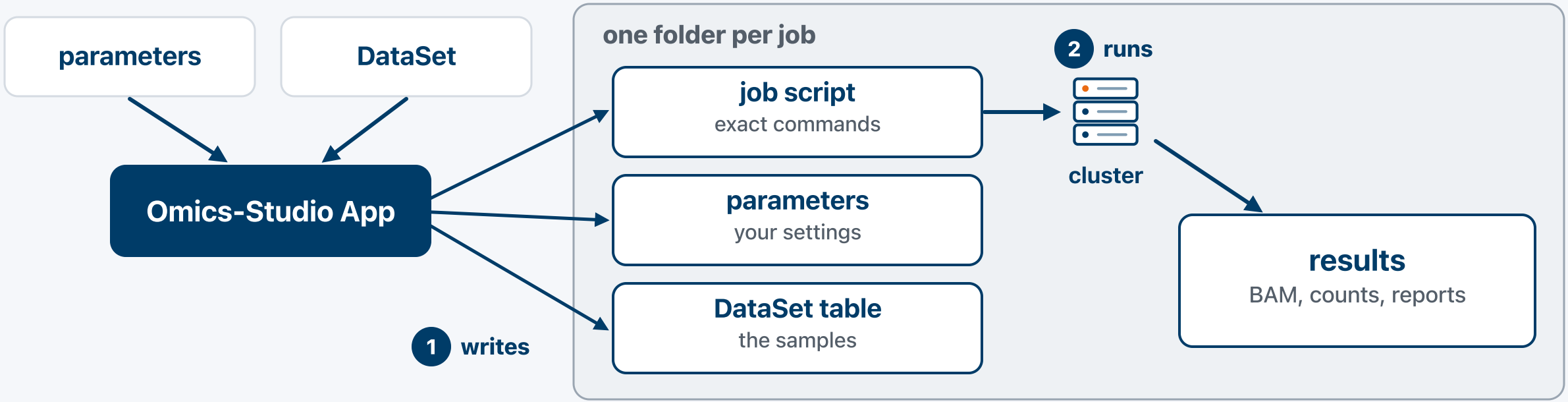


made-up values

From reads to changed genes in five steps



Omics-Studio writes the script and keeps it with the result



1 **written for you**

2 **kept together**

3 **built from metadata**

A DataSet is a table; an App makes the next one

DataSet: one row per sample

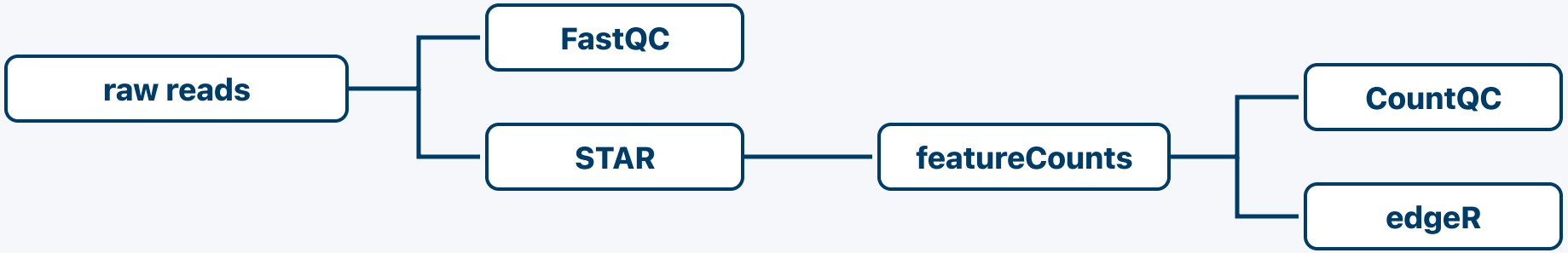
Name	Read1 [File]	Species	Condition
S1_undiff	S1_undiff.fastq.gz	Homo sapiens	undiff
S2_undiff	S2_undiff.fastq.gz	Homo sapiens	undiff
S3_undiff	S3_undiff.fastq.gz	Homo sapiens	undiff
S4_diff	S4_diff.fastq.gz	Homo sapiens	diff
S5_diff	S5_diff.fastq.gz	Homo sapiens	diff
S6_diff	S6_diff.fastq.gz	Homo sapiens	diff

STARApp
1 App = 1 job

next DataSet

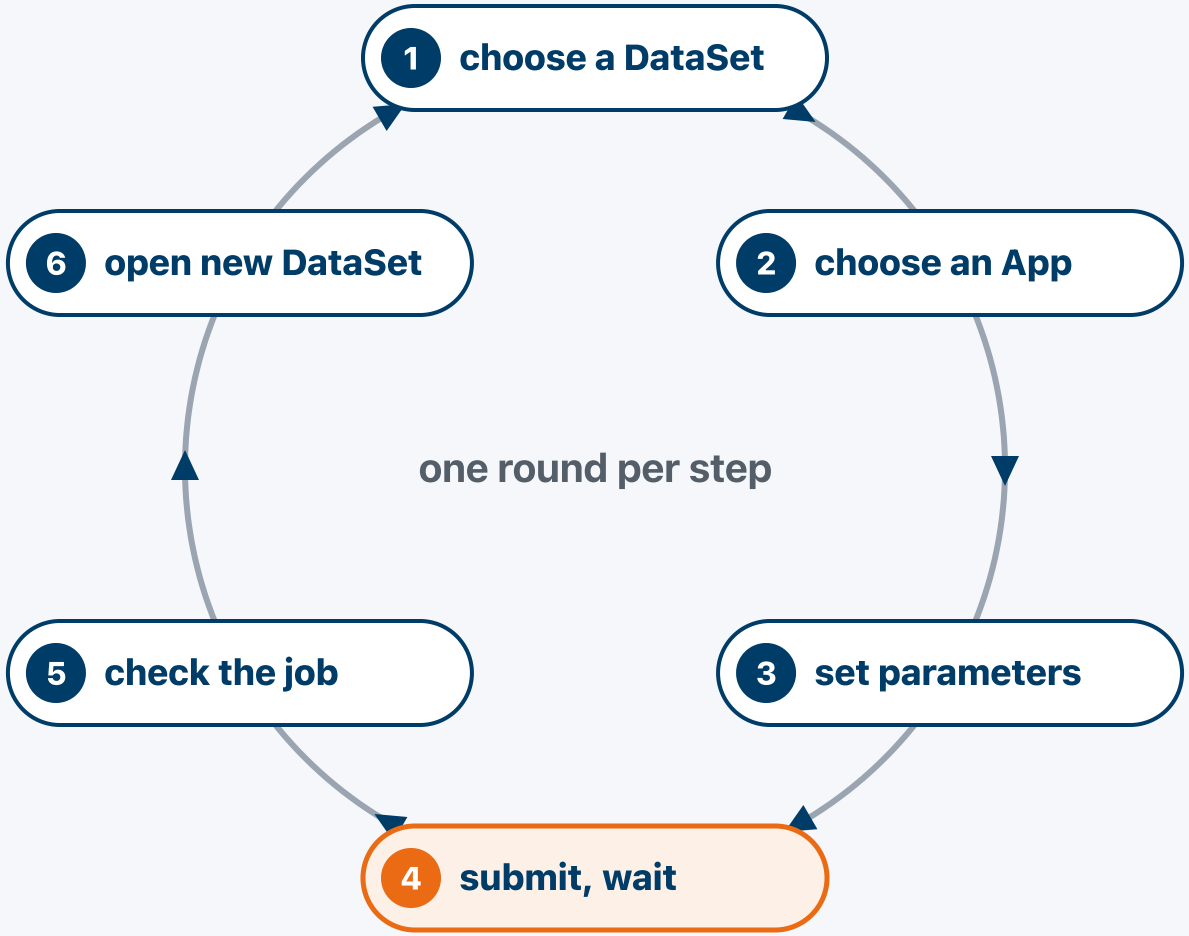
Name	BAM [File]	Condition
S1_undiff	S1_undiff.bam	undiff
S2_undiff	S2_undiff.bam	undiff
S3_undiff	S3_undiff.bam	undiff
S4_diff	S4_diff.bam	diff
S5_diff	S5_diff.bam	diff
S6_diff	S6_diff.bam	diff

DataSet tree = history of each result



Apps offered:
by the columns

Every step is the same loop: choose, run, check

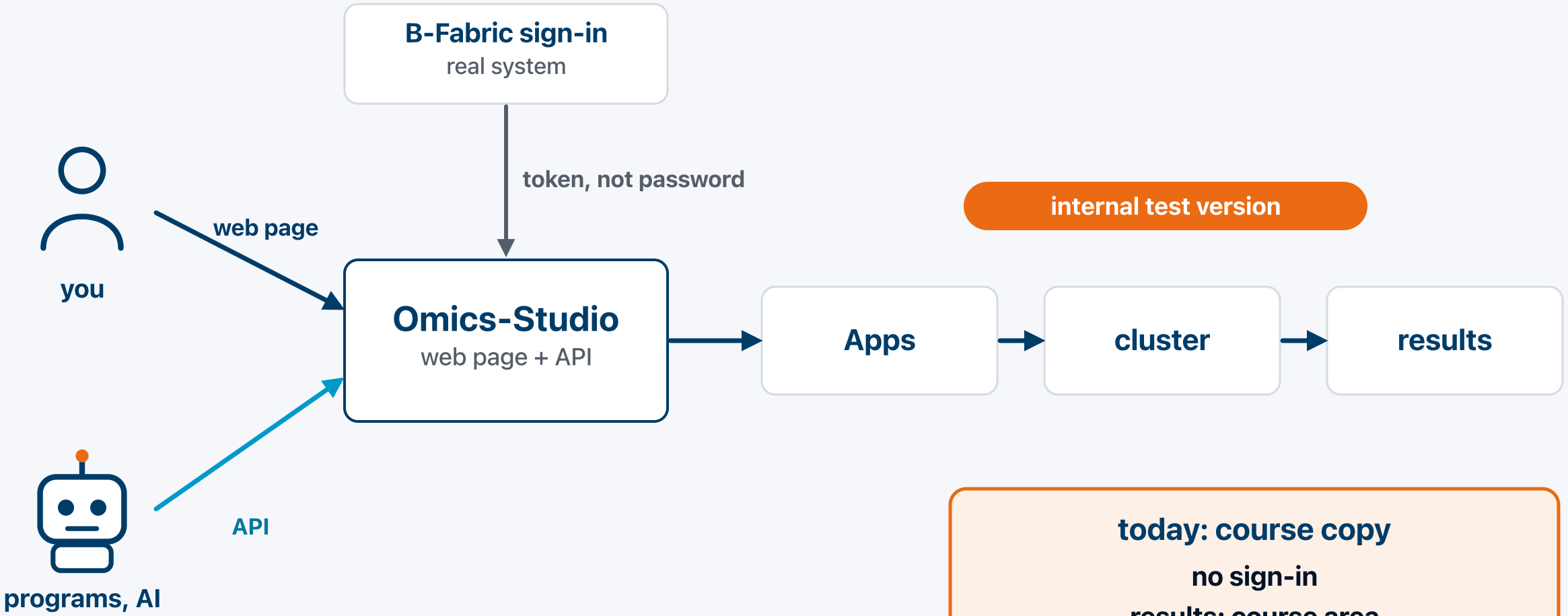


job status

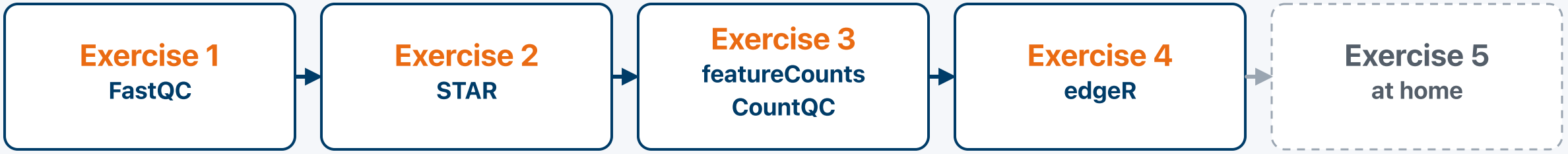
pending	in the queue
running	computing
success	new DataSet
fail	read the log

check: genome, paired-end

Omics-Studio: a web page for you, an API for programs



In the hands-on you run these steps yourself



quality reports | reads in IGV | sample groups | diff vs undiff | interactive report

Data

→ 19

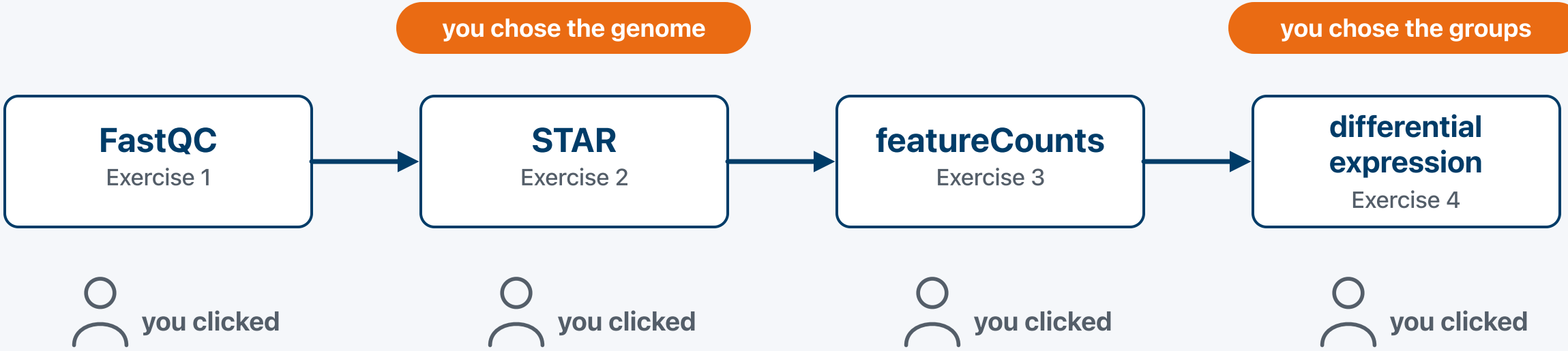
human cells, RNA-seq
3 undiff + 3 diff
same as SUSHI course

Where

course Omics-Studio
no sign-in
address: exercise page

next: OMAKASE (Part 3)

You chose every app; two steps hid a decision



~7 in 10 **real analyses:**
2 or more apps in a chain

FGCZ orders, one year, rounded

OMAKASE proposes the course; a person releases it

Omics **M**etadata **A**utomated **K**nowledge-based **A**nalysis **S**uggestion & **E**xecution

お任せ = “chef, you choose”



Rules decide · AI assists · Humans release

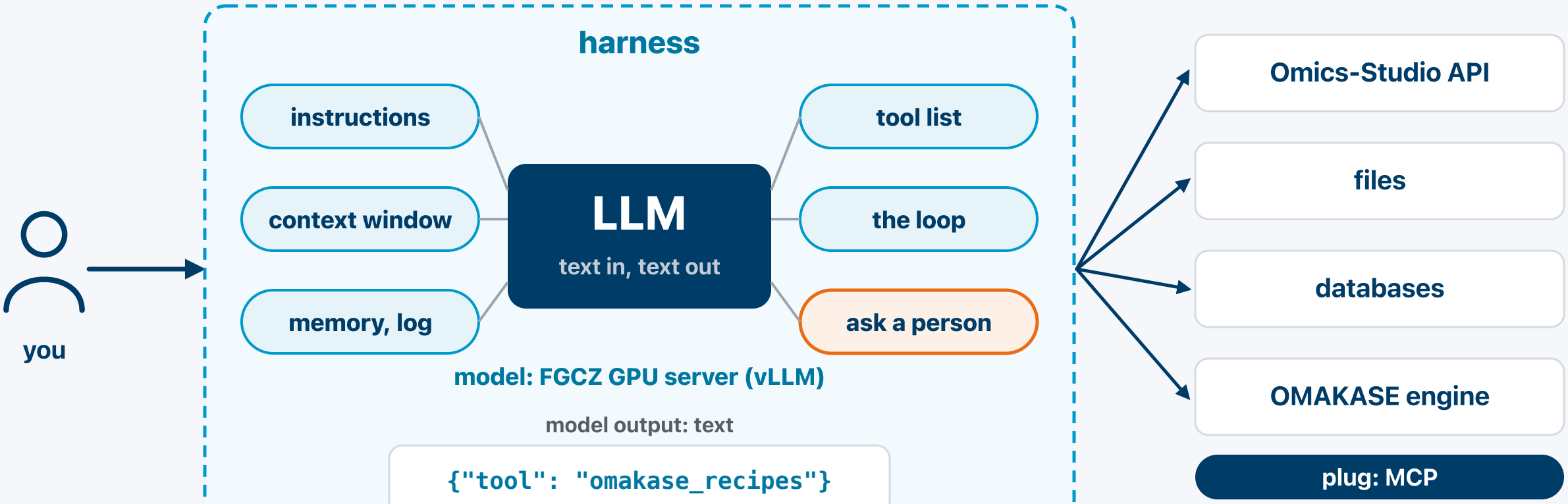
order finished	find its dataset	choose a recipe	a person approves	run the chain	tell the customer
works	works	a person	works	works	not yet

recipe = existing apps, in order

nothing runs before approval

prototype · test system

An LLM only writes text; a harness lets it act

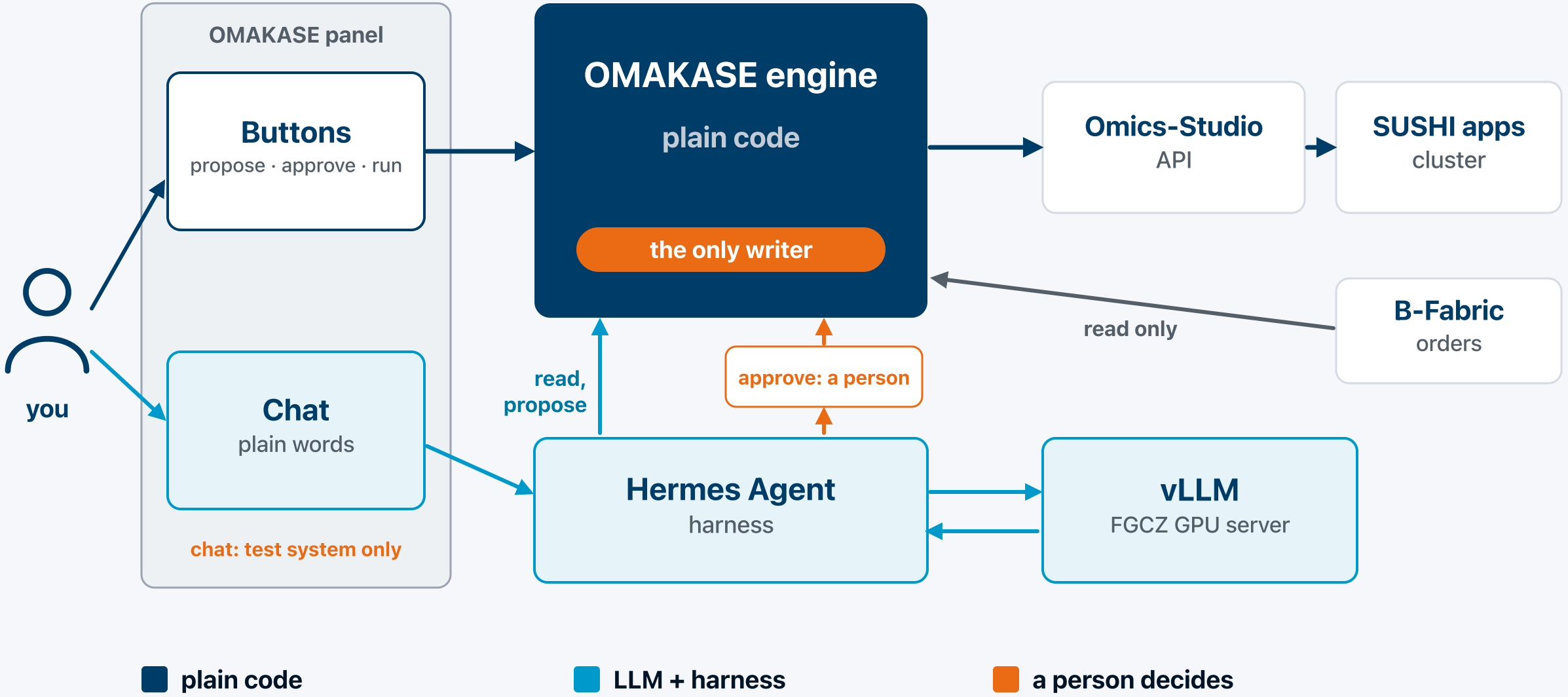


examples: Claude Code | Hermes Agent | OMAKASE loop | ChatGPT app

agent = LLM + harness + tools

click a word

Plain code runs the analysis; the LLM only proposes



Where OMAKASE sits: an AI proposes, a person runs

more freedom for the AI →	person writes the pipeline	ready-made pipelines, apps	AI picks from fixed tools	AI writes new code
person starts the run	Nextflow, Snakemake, CWL, WDL	nf-core, Galaxy, SUSHI, Omics-Studio		Co-Scientist (advice)
AI proposes, person approves			OMAKASE, nf-core skill, CRISPR-GPT	BioMaster
AI runs it, no approval stated			ClawBio, GeneAgent	Biomni, AutoBA, Robin

from papers and docs, 2026-10-22

The workflow stays; who starts each step changes

Workflow — Part 1

QC, mapping, counting,
differential expression
one file per step

Omics-Studio — Part 2

DataSet, App,
next DataSet
script kept

OMAKASE — Part 3

rules decide
AI assists
humans release

OMAKASE: not yet

automatic recipe choice

message to the customer

differential expression step

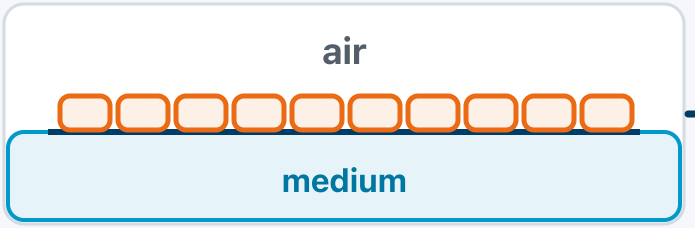
chat outside the test system

Which step would you trust to a model?

How would you check it?

Hands-on data: airway cells before and after differentiation

Primary HBECs
human bronchial epithelial cells, from human lungs



Airway epithelium
basal, ciliated, goblet and club cells

ALI culture: air on top, medium below · 30 days

before: undiff
S1_undiff · S2_undiff · S3_undiff
primary cells, like basal cells

vs

after ALI: diff
S4_diff · S5_diff · S6_diff
several airway cell types

bulk RNA-seq · 3 replicates each · course copy: 100 000 reads a sample

a model for asthma, pulmonary fibrosis, COPD and virus infections such as influenza

Which cell types the marker genes point to

cell type	role	marker genes
Basal cells like the undiff cells	stem cells, renew the lining	*TP63, ITGA6, KRT5, KRT6A, COL17A1*
Ciliated cells	cilia move mucus out	*FOXJ1, TUBB4A*
Goblet cells	make mucus	*MUC1, MUC5AC, SPDEF*
Club cells formerly Clara cells	secrete protective proteins	*SCGB1A1*

Exercise 4: find these genes in your edgeR result

Which condition looks like which cell type?

NGS data analysis needs a cluster and pipeline code

DETAILS · SLIDE 3

What a cluster is

- **Too big for a laptop**: one RNA-seq sample gives a BAM file of often more than 2 GB; many samples need many CPU hours.
- Many computers that share one storage. At FGCZ the results land in **gStore**.
- A **scheduler** (SLURM at FGCZ) keeps a queue. Each job asks for CPUs, memory and scratch disk, and waits until they are free.
- **Reproducible**: someone else, later, gets the same result from the same data. That needs the data, the steps, the parameters and the software versions.

What it does not show

- Small data can run on a laptop; the cluster is for size and for many samples at once.
- Cloud computing works the same way: a queue in front of rented computers.
- The 2 GB: it depends on the number of reads (details of slide 6).

BAM size from FGCZ's 2025 SUSHI lecture.

Scripts are exact; web tools are easier to start

DETAILS · SLIDE 4

A whole job script (bwa.sh)

```
#!/bin/bash
# work in a scratch folder on the compute node
cd /scratch && mkdir -p test && cd test
cp LOCATION_OF_REF/reference.fasta .
bwa index reference.fasta                  # prepare the reference
cp LOCATION_OF_DATA/data.fastq.gz .
bwa mem -t 8 reference.fasta data.fastq.gz > aln.sam   # map, 8 CPUs
samtools view -bT reference.fasta -o aln.bam aln.sam  # SAM to BAM
samtools sort -o aln.sorted.bam aln.bam
samtools index aln.sorted.bam              # makes .bai
cp aln.sorted.bam* LOCATION_OF_STORAGE     # keep the result

# submit it:  sbatch -p user -n 8 --mem=100G bwa.sh
```

How to read it

- Copy in, compute on the node's own disk, copy the result back to storage.
- `sbatch` hands the script to the scheduler with the CPUs and memory it needs.
- Web tools (Omics-Studio, Galaxy, the commercial CLC Genomics Workbench) are easy to start but offer only their own tools.
- Omics-Studio builds such a script from the App and your parameters with plain code, and keeps it next to the result (Part 2). It is **deterministic**: the same choices always give the same script. No AI writes it.

What it does not show

- BWA is a DNA mapper. For RNA-seq you use a spliced mapper such as STAR (details of slide 5).
- Tool versions: the script must also say which version of each tool to load.

From FGCZ's 2025 SUSHI lecture, comments added.

RNA-seq: place each read on the genome, then count

DETAILS · SLIDE 5 (1 OF 2)

What a read and a count are

- A **short read** is a short piece of sequence read from the RNA in a sample, often 50–150 letters.
- Which gene a read came from is unknown until it is placed on the genome: that is **mapping**.
- Then the reads on each gene are **counted**. More reads on a gene means more of that gene's RNA in the sample, roughly.
- Comparing the counts of the same gene between groups shows which genes change. The differential expression (DE) test also corrects for how many reads each sample got in total.

What it does not show

- The drawing has 12 reads; a real sample has millions.
- Reads that span an intron: next page.
- Reads that fit several places, or no gene: the tools have rules for them (details of slide 7).

Spliced and unspliced mapping

DETAILS · SLIDE 5 (2 OF 2)

The genome has introns; the mRNA that RNA-seq reads has them cut out



A read from the mRNA can start in exon 1 and end in exon 2. On the genome, its two halves lie far apart, with the intron between.

Spliced mapper: STAR, HISAT2



Splits the read across the intron and finds the splice junction. Use it with a genome.

Unspliced mapper: BWA, Bowtie2



The read must fit in one piece; at the intron it is cut or lost. Fine for DNA, or for a reference of transcripts (no introns).

Kallisto and Salmon go further: they assign reads to transcripts without a full alignment. Fast, but they find no new splice junctions.

Each step writes a file that the next step reads

DETAILS · SLIDE 6

File	What is in it	Made by	Read by
FASTQ	Reads and a quality letter for each base; 4 lines per read	sequencer	FastQC, STAR
FASTA	The genome sequence (the reference)	a reference release	STAR
GTF	Where genes and exons sit on the genome (the annotation)	a reference release	featureCounts
SAM	Alignments as plain text; RNA-seq: often more than 10 GB per sample	STAR (if asked)	—
BAM	The same, compressed, with an index (`.bai`) for fast lookup; RNA-seq: often more than 2 GB	STAR	featureCounts, IGV
count table	Reads per gene and sample, as text	featureCounts	CountQC, edgeR

Reading the examples on slide 6

- **FASTQ**: 4 lines per read — name, sequence, +, a quality letter per base.
- **BAM**: read, chromosome, position, and how it aligns: `19M` 19 letters match; `9M800N10M` 9 letters, an 800-letter intron (N), 10 letters.
- **count table**: one row per gene, one column per sample. **gene list**: log2 ratio +1 = twice as much, –1 = half; FDR = expected share of false hits.

What it does not show

- Sizes depend on the number of reads. As BAM, an exome is often 2–10 GB, a whole genome 100–300 GB.
- The example rows on slide 6 are made up.

Sizes from FGCZ's 2025 SUSHI lecture.

From reads to changed genes in five steps

DETAILS · SLIDE 7

Step	Tool in the exercises	Other tools	What you check
1 quality control	FastQC; MultiQC sums up all samples	fastp (also trims reads)	base quality, adapters, over-represented sequences
2 mapping	STAR (can trim with fastp first)	HISAT2; Kallisto or Salmon	share of reads mapped; the reads in IGV
3 counting	featureCounts	Kallisto or Salmon count while mapping	share of reads on genes
4 count QC	CountQC	—	do replicates cluster together? any outliers?
5 differential expression (DE) test	edgeR	DESeq2, limma	how many genes change, at which threshold

How to read the five steps

- Steps 1 and 4 are checks: they decide whether you trust the data enough to go on.
- Steps 2, 3 and 5 make new data: a BAM file, a count table, a gene list.

What it does not show

- Experimental design: the most important choices (replicates, groups, batches) are made before sequencing.
- What comes after: pathways, single-cell analysis and more.

Omics-Studio writes the script and keeps it with the result

DETAILS · SLIDE 8

One result folder

```
STAR_2025-11-24--18-46-28/   name: App + date and time
  S1_undiff.bam              the results
  S1_undiff.bam.bai          (and the other samples)
  scripts/                   the job scripts that ran
  parameters.tsv             every parameter you set
  input_dataset.tsv          the DataSet it started from
  dataset.tsv                the new DataSet
```

Omics-Studio runs the same Apps as SUSHI, its predecessor at FGCZ; the folder layout comes from the Apps' code (`sushi_fabric`), read on 2026-10-23.

Why it matters

- Someone can re-run the script years later, without SUSHI.
- A reader can see which reference and which parameters were used.
- Built from the metadata: species, groups and the reference come from the DataSet table. The script calls STAR, featureCounts and the other tools.
- Paper on SUSHI, the predecessor: Hatakeyama et al., "SUSHI: an exquisite recipe for fully documented, reproducible and reusable NGS data analysis", *BMC Bioinformatics* 17:228 (2016). doi:10.1186/s12859-016-1104-8

What it does not show

- The raw reads: the folder points to them, it does not copy them.
- The job log: it is shown on the Jobs page.

A DataSet is a table; an App makes the next one

DETAILS · SLIDE 9

Tag on a column	What it means	Example
[File]	a path to a data file	Read1 [File], BAM [File]
[Link]	a link to a report page	the FastQC and MultiQC reports
[Factor]	a grouping; the DE test compares its values	Condition [Factor]: undiff, diff
[Characteristic]	a property of the sample, kept for the record	FragmentSize, SampleConc
Species	which organism; used to pick the reference genome	Homo sapiens

Which Apps you are offered

- Every App lists the columns it needs. STAR needs `Name`, `Read1` and `Species`.
- The page shows only the Apps whose columns the DataSet has. So reads offer FastQC and STAR; a count table offers CountQC and edgeR.

What it does not show

- A wrong value in a column: the App runs anyway. A wrong Species gives the wrong genome.

From SUSHI's code: `STARApp.rb` (`@required_columns`) and the App list filter, read on 2026-10-23.

Every step is the same loop: choose, run, check

DETAILS · SLIDE 10

Parameter	What it sets
name	the job's name; also the result folder's name
cores, ram, scratch	CPUs, memory and temporary disk to reserve on the cluster
samples	which rows of the DataSet to use
refBuild (STAR)	the reference genome and its annotation
paired (STAR)	true if each fragment has two reads (Read1 and Read2)
strandMode (STAR)	which DNA strand the reads come from; it depends on the library kit

Job states and the form

- The form is filled in for you; the values are suggestions. For STAR, check the genome and whether the reads are paired-end.
- **pending**: the script is being written, or the job waits in the queue.
- **running**, **success**: the new DataSet appears when it is done.
- **fail**: open the job's log. Its last lines usually say what went wrong.

What it does not show

- Every parameter: each App has its own. The form shows a short help text next to many of them.

Omics-Studio: a web page for you, an API for programs

DETAILS · SLIDE 11 (1 OF 2)

Part	What it does	Built with
Web page	What you click	Next.js (a web framework)
API	The door for programs: `/api/v1/projects`, `/api/v1/jobs`, … (next page)	Rails 8
Sign-in	Real system: through B-Fabric; a program receives a time-limited token, never the password. Course copy: no sign-in	B-Fabric (FGCZ's data system)
Permission to submit	A separate tick at sign-in. Without it, reading works and submitting returns `403 insufficient_scope`	a "scope" on the token
Apps, cluster, results	The same SUSHI apps, job manager and SLURM cluster. The course copy writes results to a course area	shared with legacy SUSHI

What it shows

- The new part is the web page and the API; the analysis machinery is the same.
- An AI agent can use SUSHI through the API without ever holding a password.

What it does not show

- Every legacy feature: most daily ones work, some are still missing (stopping a running job, running on part of the samples).
- That it is open to all FGCZ users: today it is an internal test version.

API and tokens: how a program uses Omics-Studio

DETAILS · SLIDE 11 (2 OF 2) · API

An API: a door for programs

- A program sends a request to an address and gets data back, without a web page.
- On the real system a program signs in without a password: it shows a short code, you approve it in B-Fabric, and the program receives a **token** that works for a limited time.
- The right to submit jobs is a separate **scope** on the token. Without it, the API answers `403 insufficient_scope`.
- So an AI agent such as Claude Code can read datasets and submit jobs without ever holding a password.

What a request looks like

```
curl -H "Authorization: Bearer $TOKEN" \
     <OMICS_STUDIO_API>/api/v1/projects

curl -X POST -H "Authorization: Bearer $TOKEN" \
     -H "Content-Type: application/json" \
     -d '{"job": {"dataset_id": 12345,
          "app_name": "Fastqc",
          "parameters": {"cores": 1}}}' \
     <OMICS_STUDIO_API>/api/v1/jobs
```

The first lists your projects; the second submits FastQC on a dataset. OMAKASE submits its jobs through this same door.

You chose every app; two steps hid a decision

DETAILS · SLIDE 13

What was counted

- Every FGCZ sequencing order finished over one year, read from the job history of SUSHI.
- Orders that received only quality control were set aside; the rest are the “real analyses”.
- Of those, about 7 in 10 ran two or more apps one after the other.

What it does not show

- That the chains were the same: they were not (appendix A3).
- How long a person waited between the steps.
- Which choices were hard: the genome and the comparison groups are the two you made yourself.

The numbers in this deck about FGCZ orders are rounded on purpose.

OMAKASE proposes the course; a person releases it

DETAILS · SLIDE 14

Stage	What happens	Today
1 order finished	A watcher asks B-Fabric for the whole list of finished orders at each check and subtracts the ones it already handled. Read only.	works
2 find its dataset	Finds the raw dataset that carries the order number; declines if there is none or more than one.	works
3 choose a recipe	Recipes are text files written by a bioinformatician. Automatic choice declines today, so a person names one.	a person names it
4 a person approves	The proposal shows every step, every parameter and the genome. Nothing is submitted before this.	works
5 run the chain	Submits independent steps together, waits for each, stops at the first failure.	works
6 tell the customer	An e-mail is written to a file; sending it is switched off.	not yet

OMAKASE = **O**mics **M**etadata **A**utomated **K**nowledge-based **A**nalysis **S**uggestion & **E**xecution. The name is provisional. omakase (お任せ) = “I leave it to you, chef”: the chef proposes the course. OMAKASE writes no new apps: a recipe is existing Omics-Studio Apps put in order (appendix A6). Nothing runs before a person approves. How the genome is chosen, why a person approves, how the chain runs: appendix A2–A4. Top-right image: the cover of FGCZ’s OMAKASE deck of 2026-09-16, resized.

An LLM continues text, one token at a time

DETAILS · SLIDE 15 · LLM AND TOKEN

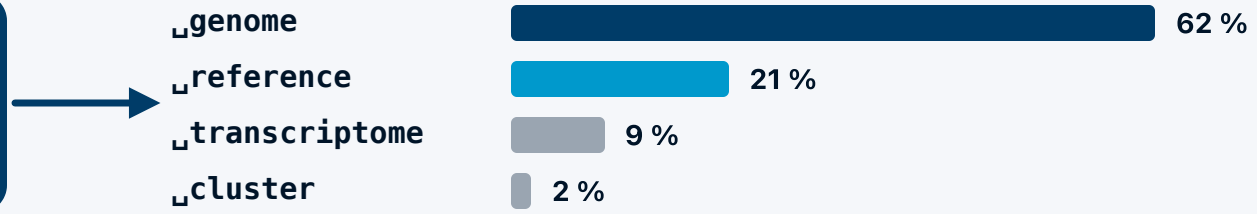
1 The text is cut into tokens

`RNA` `-seq` `␣reads` `␣are` `␣mapped` `␣to` `␣the` `?`

how one model might cut it; ␣ is a space.
Real cuts differ between models.

2 The model scores every possible next token

LLM
a very large network



made-up numbers, to show the idea

3 It adds one token and starts again

`RNA-seq reads are mapped to the` **genome** ← then again, one token at a time, until it stops

Facts

1 token is about 4 characters of English (¾ of a word).
Price, speed and limits are counted in tokens.
It writes text only. When an LLM "does" something,
a program around it did it (the harness, slide 15).

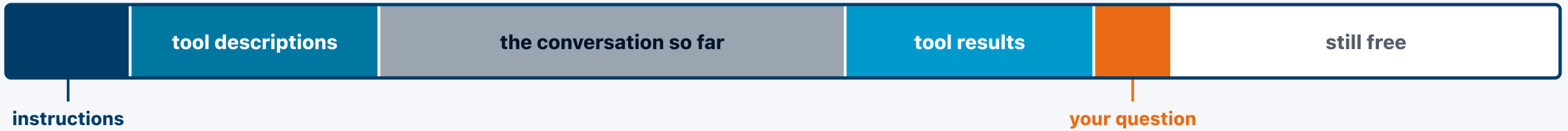
Hallucination

The most likely next words are not always true.
Example: a confident name of a genome build that
does not exist. That is why OMAKASE lets the
model pick only from a closed list (appendix A2).

The context window is all the model sees

DETAILS · SLIDE 15 · PROMPT AND CONTEXT WINDOW

Everything the model reads in one turn has to fit in its context window



a drawing: the shares are not measured

the limit: 262,144 tokens for the model on FGCZ's test system

Tool descriptions are paid for on every turn

all 100 tool descriptions — about 22,800 tokens = 8.7 % of the window

16 tools + one "search for a tool" tool — about 2,900 tokens = 1.1 %; the rest are found when needed

Prompt

The text you add. A good prompt says what is wanted, gives the data, and says what to do when the answer is unknown ("say UNKNOWN", as in appendix A2).

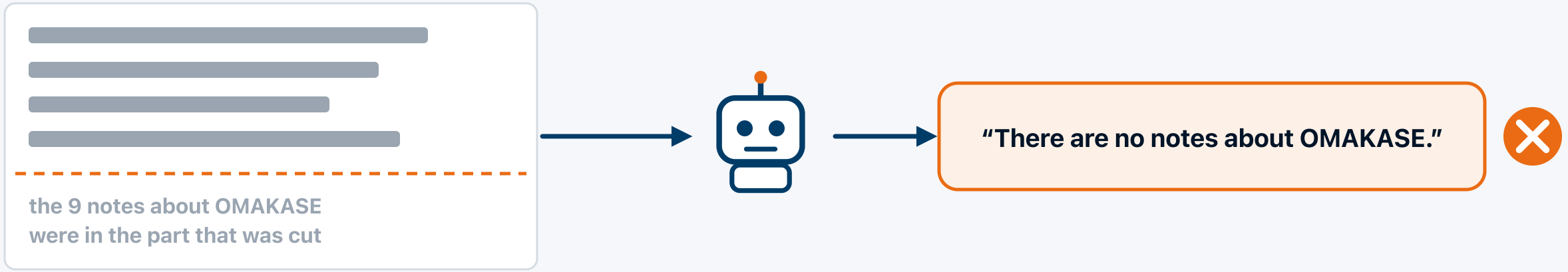
The model keeps nothing between turns

The harness sends the conversation again with every turn. Long chats get slower and cost more, and what does not fit is cut (next page).

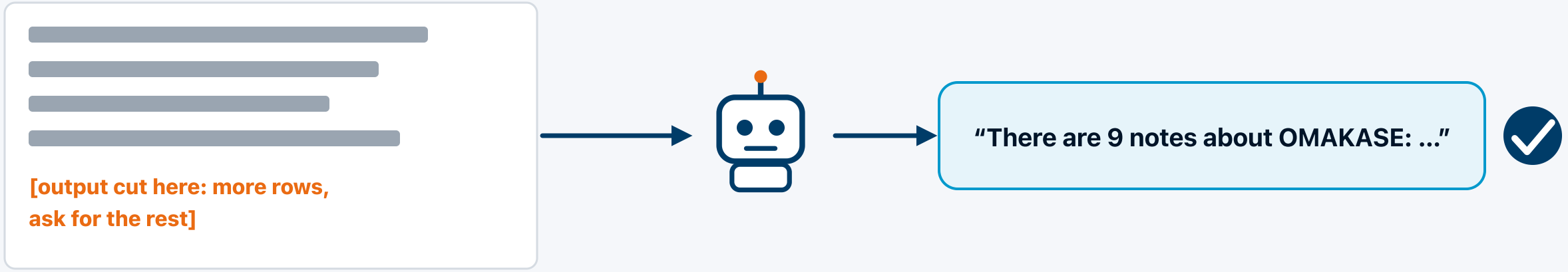
A model trusts what it is shown, even a cut-off list

DETAILS · SLIDE 15 · WHEN THE CONTEXT IS CUT

Before: the list was cut without a word



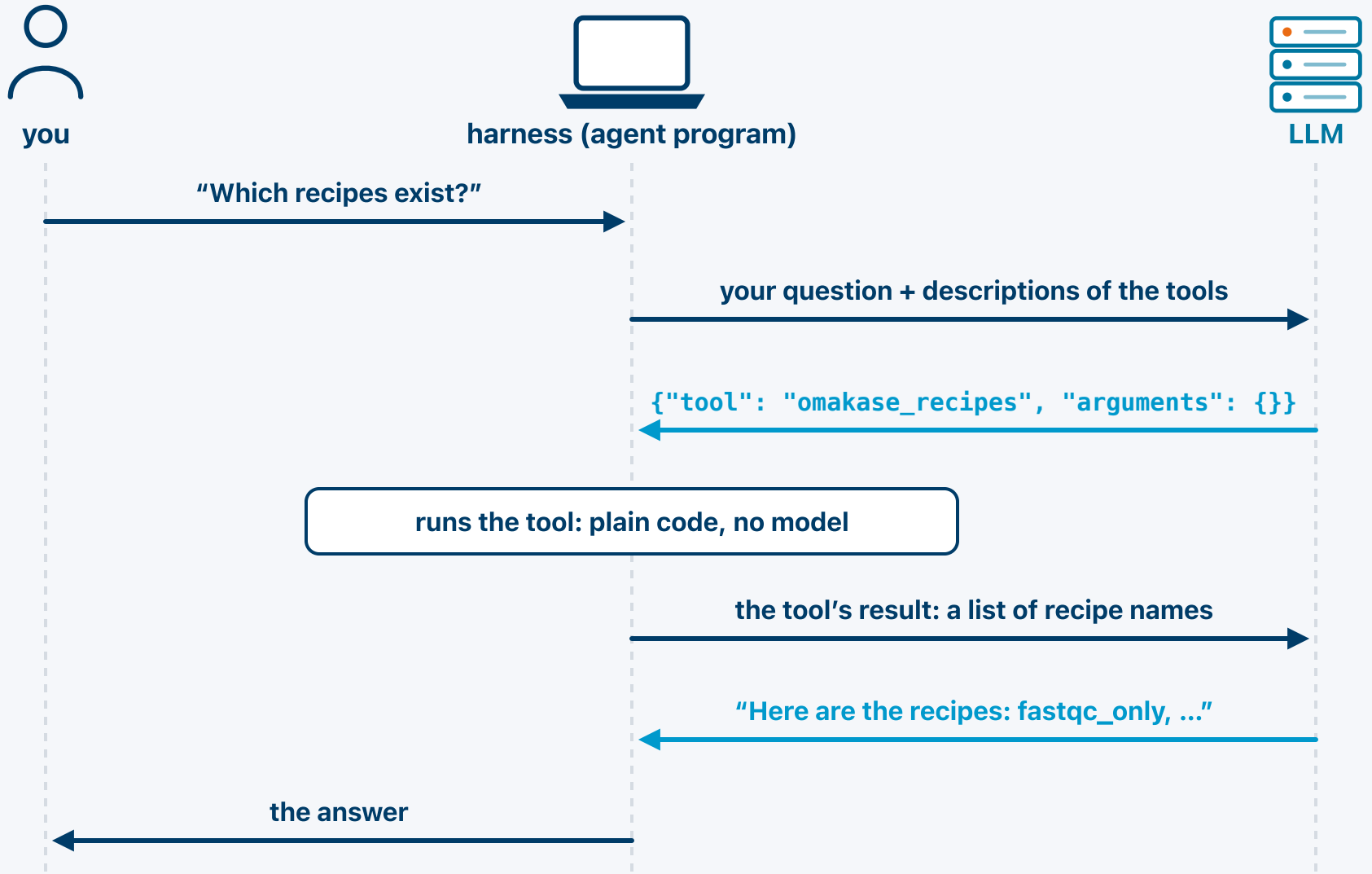
After: the cut is announced



The model cannot see what is missing. Tell it when its input is incomplete — and count important things yourself.

Tool calling: the model asks, a program does

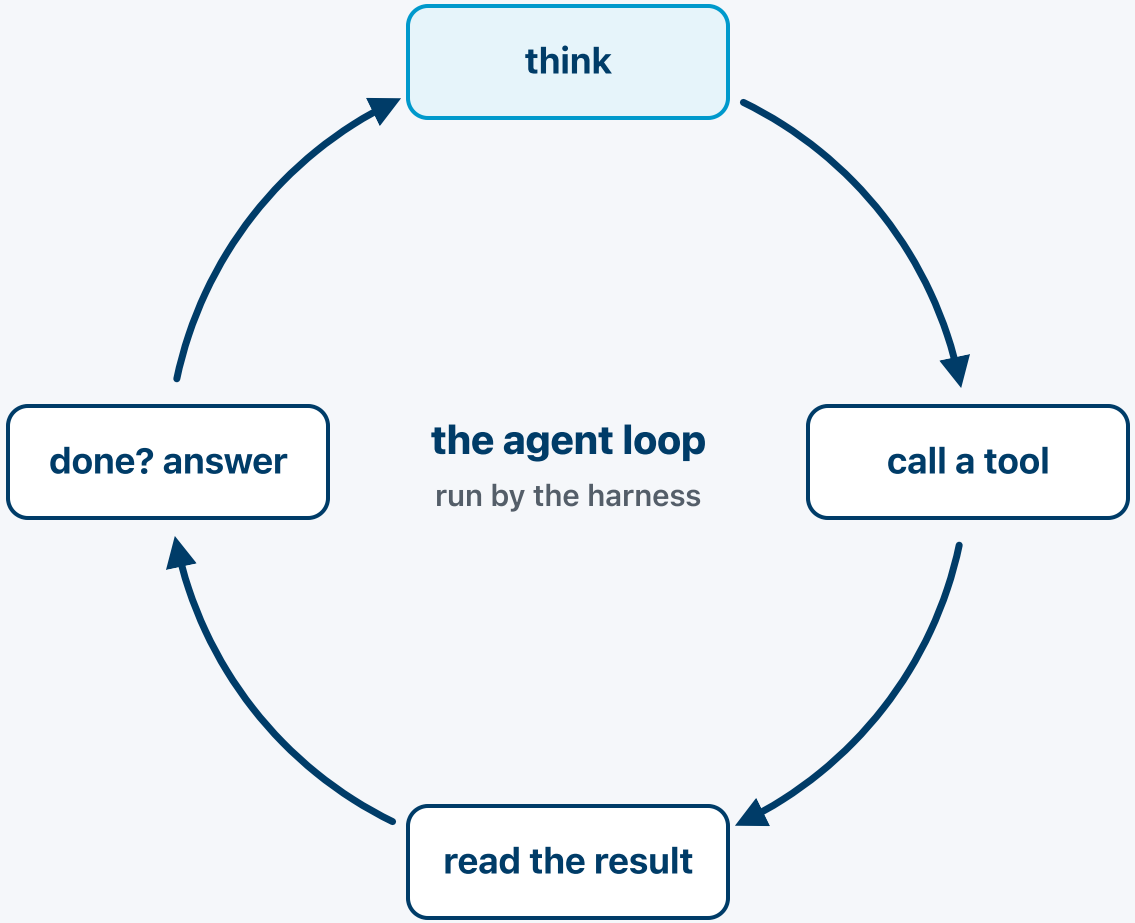
DETAILS · SLIDE 15 · TOOL CALLING



The model only writes the request (cyan). The harness decides whether to run it, and runs it.

An agent is an LLM in a loop with tools

DETAILS · SLIDE 15 · AGENT



Thin loop (the OMAKASE panel's own)

About 100 lines of Python around the model.
Shows 16 tools plus a search tool (details:
context window). Fast; no memory between chats.

Hermes Agent

An open-source agent program by Nous Research.
Adds memory across chats, skills, scheduled
tasks, and chat apps such as Slack.
Speaks the same API as OpenAI's models, so
other programs can call it. More tokens per turn.

A harness is the program around the model

DETAILS · SLIDE 15 · HARNESS



	Claude Code	Hermes Agent	OMAKASE's thin loop
Where it runs	your terminal	a server, behind a chat	inside the OMAKASE panel
Tools	files, shell commands, MCP servers	about 40 of its own, plus MCP	16, plus a search tool
Memory	notes it reads from the project (CLAUDE.md)	across chats	none between chats
Asks a person before	editing files or running commands, as set up	not compared here	approve and run: always
One test question	not measured	8 s (60 s from a cold start)	1.2 s

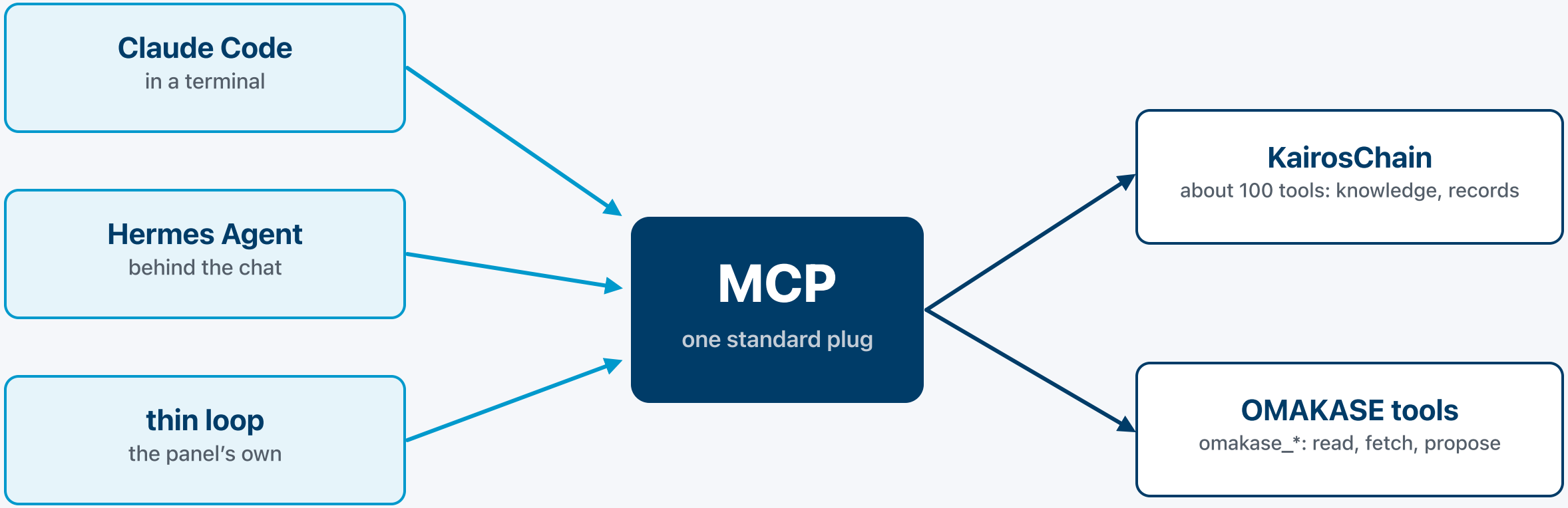
Why the word "harness"

- Like a horse's harness: the horse gives the power; the harness decides what it pulls, and where.
- The same model behaves differently in different harnesses. A bigger harness gives more features and reads more tokens per turn.

Times: one question to the same model on FGCZ's test system, September 2026; answer quality was not compared.

MCP: one plug between agents and tools

DETAILS · SLIDE 15 · MCP



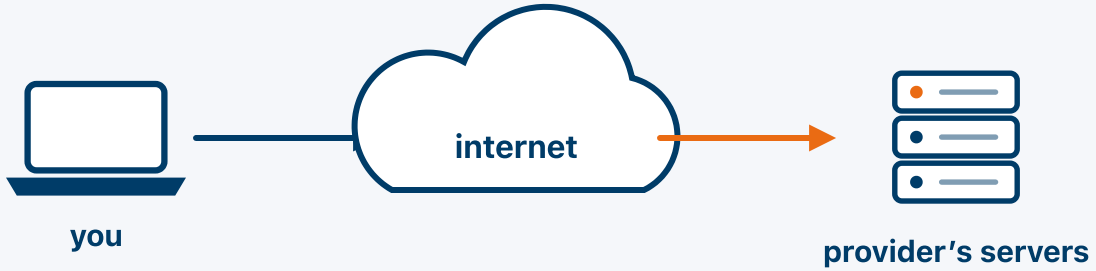
Like USB for tools: any harness that speaks MCP can use any MCP server's tools.

The server lists its tools with a description of each; the harness shows those descriptions to the model.

Where the model runs: hosted or on-premises

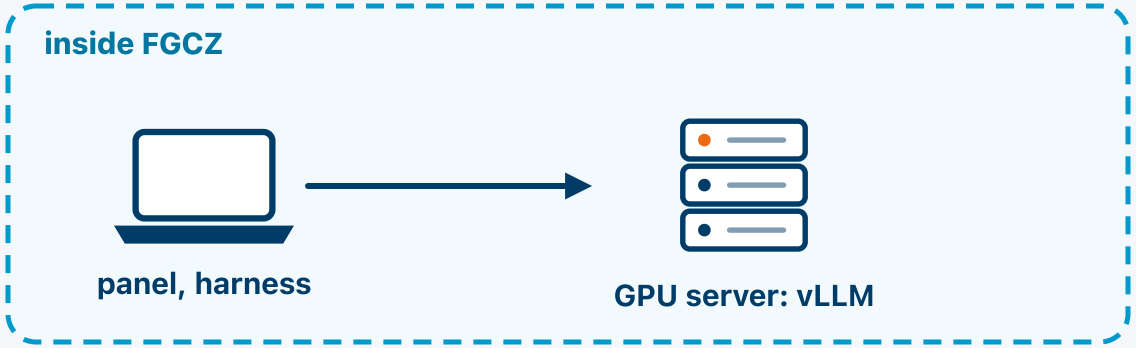
DETAILS · SLIDE 15 · VLLM AND ON-PREMISES

Hosted: the model runs at a provider



your text travels to the provider

On-premises: the model runs at FGCZ



order data stays inside FGCZ

vLLM

Open-source software that serves ("runs") a model on GPUs you own. It speaks the same API as OpenAI's models, so the same client code works with a hosted or a local model.

Open-weight model

Its trained parameters can be downloaded and run locally, unlike most hosted models. The model served can change; programs ask the server which one it has.

In OMAKASE

The chat and the genome suggestion use only the model at FGCZ. One shared GPU server: many users at once means waiting, not a bug.

Plain code runs the analysis; the LLM only proposes

DETAILS · SLIDE 16

Part	What it does	Kind
Panel	One web page. Its buttons run the engine's command line — the same commands a person types	plain code
OMAKASE engine	Order watcher, recipes, proposals, chain runner; keeps its own record (a small database)	plain code — **the only writer**
Hermes Agent	The harness behind the chat (details of slide 15: agent)	LLM side
vLLM	Serves an open-weight model on FGCZ's GPU server (details of slide 15: vLLM)	LLM side
MCP tools	`omakase_*` tools (read, fetch an order, propose) and about 100 KairosChain tools	approve / run become requests
Omics-Studio API	Receives the job submissions	plain code
B-Fabric	Where orders are recorded; OMAKASE only reads it	read only

Why the buttons skip the model

- A run must be repeatable: the same order and recipe must give the same jobs. Plain code does that; an LLM does not promise it.
- The panel shows both halves on one page on purpose, so the difference is visible.
- Through the chat, the agent can read and propose directly; approve and run become requests, and a person clicks Allow.

What it does not show

- The production system: there the chat is switched off.
- Claude Code can also drive the same command line from a terminal.

Where OMAKASE sits: an AI proposes, a person runs

DETAILS · SLIDE 17 (1 OF 2) · WORKFLOW SYSTEMS

System	What it is	Who decides the steps	Size, if stated
Nextflow	A language and engine for writing pipelines as code	a person writes it	—
nf-core	A community collection of Nextflow pipelines built from one template	the community curates	**159** pipelines, 106 released (2026-10-22)
Snakemake	Python-based “rules” from input to output files; it works out the order	a person writes it; the Catalog lists GitHub workflows without checking	Catalog: 6,481 repositories, 472 “standardized”
Galaxy	A web platform: run tools and workflows by clicking	the user; IWC keeps peer-reviewed workflows	650,000+ users, ~11,000 tools (2026 paper)
CWL	An open standard describing tools and how they connect; many engines run it	a person writes it	—
WDL	A readable workflow language from the Broad Institute; run by Cromwell or miniwdl	a person writes it	—

What they share

- Reproducibility comes from pinning: containers or Conda fix the software, versioned releases fix the pipeline.
- A person still decides which pipeline runs on which data, and starts it.

What it does not show

- Quality: a pipeline’s place in the table says nothing about how good its results are.
- 2026 AI add-ons: Seqera AI can also launch runs; its default approval setting is not documented.

2025–26: Seqera (for Nextflow) and Galaxy 26.1 added MCP servers, so outside AI agents can start runs; Galaxy’s is off by default. Slide 17 places each system from its paper or documentation (2026-10-22); “no approval stated” means none is described, not that none can be added. Links: appendix A7.

Where OMAKASE sits: an AI proposes, a person runs

DETAILS · SLIDE 17 (2 OF 2) · AI AGENTS FOR BIOLOGY

Agent	How it acts	Does a person approve before it runs?
Biomni (Stanford)	writes and runs its own code	not stated; its README warns it "executes LLM-generated code with full system privileges"
ClawBio (Corpas)	calls versioned skills: "a contract, not a prompt"	depends on the host agent (Claude Code, OpenClaw, …)
BioMaster (HKUST-GZ)	writes and runs commands; separate plan / run / debug / check agents	optional: confirm the plan once; "auto-accept" exists
AutoBA (KAUST)	writes and runs code, repairs its own errors	no: "requires no human intervention"
Robin (FutureHouse)	proposes hypotheses; an analysis agent writes notebook code	people run the experiments; no step stated for the code
Co-Scientist (Google)	generates and debates hypotheses	nothing runs: advice only
CRISPR-GPT (Stanford)	fixed task steps for gene-editing design, calling tools	yes: the user monitors and corrects each step
GeneAgent (NIH)	checks its claims about gene sets against fixed databases	no: fully automated, but only reads databases
nf-core skill (Anthropic)	launches pinned nf-core pipelines from Claude Code	yes, as a sentence in the prompt: "DECISION POINT: Confirm with user"

Compare numbers only on the same test

- AutoBA reports up to 35 of 40 cases (87.5 %) end to end on its authors' own cases. On BioMaster's benchmark it completed 13 of 49 (26.5 %); BioMaster completed 47 of 49 on its own benchmark.

From each paper or repository, checked 2026-10-22 (sources in appendix A7). The quotes are copied from the projects' own text.

Appendix A1 · Words in this lecture

LLM → 15 det
Large language model: it writes text, one token at a time; it runs nothing.

Token → 15 det
The unit a model reads and writes: about 4 characters of English.

Hallucination → 15 det
A fluent answer that is not true, e.g. a genome build that does not exist.

Prompt → 15 det
The text you send to the model, with instructions, data and examples.

Context window → 15 det
All the model sees in one turn; what does not fit is cut.

Tool calling → 15 det
The model replies “call tool X with these values”; a program runs it.

Agent → 15 det
An LLM in a loop with tools: think, call a tool, read the result, repeat.

Harness → 15 det
The program around the model: prompt, tools, loop, permissions, memory.

Hermes Agent → 15 det
An open-source harness with memory, schedules and chat apps built in.

MCP → 15 det
Model Context Protocol: one standard plug that lets any agent use tools.

vLLM → 15 det
Open-source software that serves an open-weight LLM on your own GPUs.

On-premises → 15 det
Runs on FGCZ’s own computers, so the data does not leave FGCZ.

API → 11 det
A door for programs: send a request, get data back, no web page needed.

Deterministic → A5
Same input, same output, every time. Plain code is; an LLM usually is not.

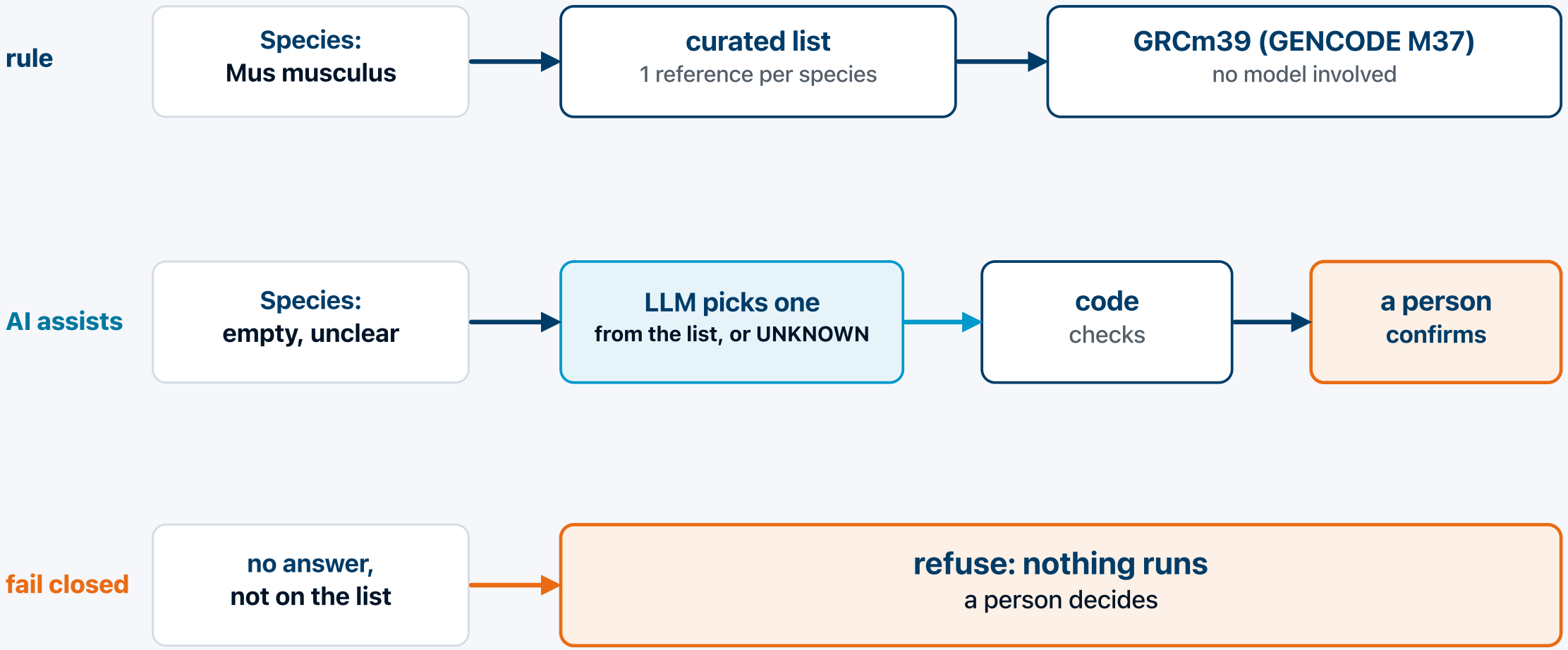
Recipe → A6
A text file: which Apps run, in which order (OMAKASE).

Click a card for its page, or press ↑ and type the word.

back to slide 15

The genome is looked up; the LLM may only suggest

APPENDIX A2



shown to the model: species fields only

The genome is looked up; the LLM may only suggest

APPENDIX A2 (2 OF 2)

The order of the checks

- 1. The dataset's **Species** column names one species on FGCZ's curated list → its reference is used.
- 2. Otherwise, the species on the order's B-Fabric samples.
- 3. Otherwise, the LLM is asked to pick **one name from the list, or UNKNOWN**.
- Code then checks: the name is on the list, the reply came from the expected model on an FGCZ server.
- A person must confirm the suggestion before the chain may start.
- What the model is shown: the species fields, the sequencing application, probe-set names. Never sample names or customers' free text.

What it does not show

- How often step 3 is needed: about half of FGCZ's raw datasets name a curated species in step 1.
- Whether the model's suggestions are good: step 3 was added in October 2026 and is a prototype.
- Datasets with two species: there is no single reference, so OMAKASE refuses.

Why a closed list? A free answer could be a genome build that does not exist (a hallucination). A name on a list can be checked by code.

Past orders do not agree on one right analysis

APPENDIX A3

~1 in 5

most common past chain was the right one

> 100 different chains

standard sequencing orders



drawing

no clear match: no proposal

a person names the recipe

FGCZ orders, one year, rounded

Past orders do not agree on one right analysis

APPENDIX A3 (2 OF 2)

What "~1 in 5" means

- For each kind of order (service type), take the chain of apps used most often in the past.
- Ask: would proposing that chain have been right for each order? About one time in five, yes.
- For standard sequencing orders it was right less than one time in ten: they used more than 100 different chains.
- This simple rule is the bar any automatic choice has to beat.

What it does not show

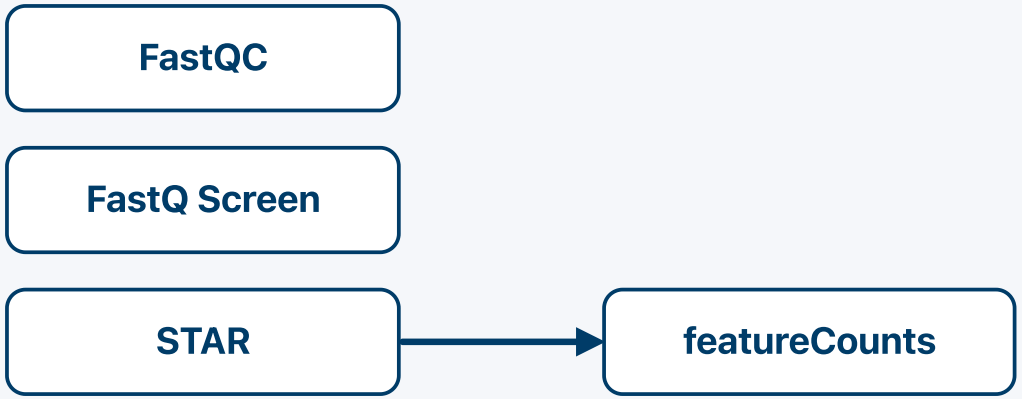
- That a trained model would do as badly: no model has been trained or tested on this yet.
- That the data are bad: real analyses simply differ.
- A test with fixed rules, written before any results, will decide later whether an automatic choice is good enough.

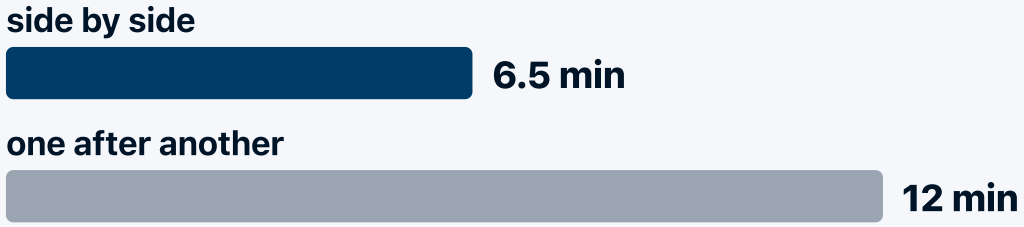
Counted from FGCZ's job history over one year. Numbers rounded.

Run side by side; stop at the first failure

APPENDIX A4

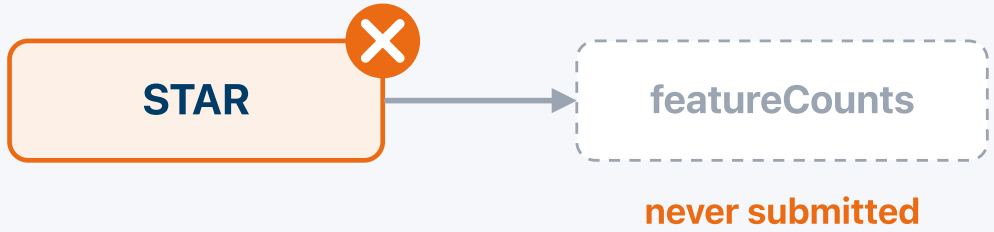
Side by side





test system, 2 mouse samples

Stop at the first failure



needs STAR's output

chain stops

failure recorded

Run side by side; stop at the first failure

APPENDIX A4 (2 OF 2)

The run on the previous page

- FastQC, FastQ Screen, STAR, then featureCounts, on 2 mouse samples, on FGCZ's test system (September 2026).
- Side by side: 6 min 31 s from start to finish.
- The same jobs one after another would have taken 12 min 13 s.
- The recipe says which step waits for which (appendix A6).
- featureCounts needs STAR's output; starting it anyway would only fail again. So OMAKASE stops the chain and records which step failed and why.

What it does not show

- Real datasets: with more samples, STAR takes much longer and the saving changes.
- Every failure stops the chain: a short cluster hiccup is retried once before the chain stops.
- Undoing work: jobs already running when another step fails keep running.

Appendix A5 · Why not let the LLM run the whole analysis?

The same: an LLM could read an order and name a recipe. That part is real.

	Plain code + a person	Letting the LLM decide
Same input, same output?	yes, every time	not promised: a small change in the wording can change the answer
How often right?	says no when unsure	the simple past-majority rule: ~1 in 5 (appendix A3)
Can it be checked?	a closed list, checked by code	a free answer may name something that does not exist
Cost per decision	milliseconds, no tokens	seconds and thousands of tokens (slide 15 details)
Who released the run?	a named person clicked Approve	"the model did it" explains nothing afterwards

So the LLM is used where a wrong answer is cheap and gets checked: talking, drafting, suggesting.

back to slide 16

What a recipe file looks like

APPENDIX A6

A recipe, shortened

```
id: rnaseq_meeting_shape
version: 2
steps:
  - id: fastqc
    app: FastqcApp
    after: []            # starts with the chain
  - id: fastqscreen
    app: FastqScreenApp
    after: []
  - id: star
    app: STARApp
    after: []
    params:
      refBuild: "{{ reference_for(species) }}"   # looked up (appendix A2)
  - id: featurecounts
    app: FeatureCountsApp
    after: [star]        # waits for STAR only
match:
  service_type: []        # empty: never chosen automatically
```

How to read it

- A recipe is a text file (YAML). A bioinformatician writes it and signs it off; an AI may only draft one.
- `after` says which step waits for which. Empty means “start with the chain”.
- The genome is not written in: it is filled in from the dataset’s Species before a person approves.
- There is no differential-expression step: choosing the comparison groups is a judgement nothing here can make.
- A recipe has a version. A proposal records which version it used, so a result can be traced back.

back to slide 14

Links: workflow systems and AI agents for biology

APPENDIX A7

Workflow systems

Nextflow — nextflow.io · code · paper 2017

nf-core — nf-co.re · paper 2020 · update 2025

Snakemake — snakemake.github.io · catalog · paper 2021

Galaxy — galaxyproject.org · code · paper 2026

CWL — commonwl.org · cwltool · paper 2022

WDL — openwdl.org · miniwdl · Cromwell

AI add-ons — Seqera AI · Seqera MCP · Galaxy 26.1

SUSHI — paper 2016

AI agents for biology

Biomni — code · Science 2026 · preprint

ClawBio — clawbio.ai · code · preprint 2026

BioMaster — code · Patterns 2026

AutoBA — code · Adv Sci 2024

Robin — code · Nature 2026

Co-Scientist — blog · Nature 2026

CRISPR-GPT — code · Nat Biomed Eng 2025

GeneAgent — code · Nat Methods 2025

nf-core skill — code

Checked 2026-10-22: 38 of 40 links opened (the SUSHI paper link was added on 2026-10-23). The Science and Advanced Science pages refuse automatic checks; both DOIs are registered. back to slide 17